

FedShop-Go: Um Motor de Consultas SPARQL Federadas Mínimo Avaliado com o Benchmark FedShop

Pedro Henrique Honorio Saito¹[DRE: 122149392]

¹ Universidade Federal do Rio de Janeiro, Ilha do Fundão, Rio de Janeiro, Brasil

Abstract. Consultas SPARQL sobre federações com centenas de endpoints heterogêneos exigem estratégias eficientes de seleção de fontes e planejamento de execução. Este trabalho apresenta o **FedShop-Go**, um motor de consultas SPARQL federadas standalone implementado em Go e avaliado com o benchmark FedShop sobre federações de 20 a 200 endpoints. Complementarmente, desenvolvemos o **fedshop-py**, um pipeline reproduzível em Python que substitui o Snakemake original do FedShop como orquestrador. A avaliação compara seis engines — RSA, FedShop-Go, FedX, PyFedX, SPLENDID e SemaGrow — em 12 templates de consulta BSBM, revelando que o FedShop-Go supera o FedX em até 80% nas consultas de domínio único (q06–q12) com tempo médio de 7,19 s, mas enfrenta timeouts nas consultas de domínio cruzado com alto volume intermediário (q02, q05). A seleção de fontes via consultas ASK consome aproximadamente 13% do tempo total, confirmando sua baixa sobrecarga relativa.

Keywords: SPARQL Federado · FedShop · Seleção de Fontes · Benchmark · Go

1 Introdução

Consultas SPARQL federadas permitem que um cliente interroge múltiplos repositórios RDF simultaneamente, tratando a federação como uma fonte de dados unificada [1]. À medida que a Web de Dados Ligados cresce, federações com dezenas ou centenas de endpoints tornam-se comuns, introduzindo desafios fundamentais: quais endpoints contêm dados relevantes para cada padrão de tripla? Em que ordem executar os padrões para minimizar a transferência de dados intermediários? Como garantir robustez diante de endpoints lentos ou instáveis?

O benchmark **FedShop** [2] foi proposto para medir escalabilidade de motores federados em cenários próximos à realidade, com federações que crescem de 20 a 200 endpoints e 12 templates de consulta derivados do Berlin SPARQL Benchmark (BSBM) [3]. Apesar de seu valor, reproduzir e estender experimentos com o FedShop original requer uma cadeia de dependências complexa baseada em Snakemake, dificultando experimentos ágeis.

Este trabalho apresenta três contribuições principais:

- **FedShop-Go:** motor standalone de consultas SPARQL federadas implementado em Go, sem dependências de RDF4J, Jena ou outros frameworks de grande porte. O motor implementa seleção de fontes por consultas ASK, planejamento por ordenação de triplas, joins por hash e por vinculação (*bind join*), e emite artefatos compatíveis com o FedShop.
- **fedshop-py:** pipeline Python reproduzível que substitui o Snakemake do FedShop, expondo as etapas de geração, ingestão, instanciação de consultas, avaliação e coleta de métricas como CLI e API programável.
- **Avaliação comparativa:** execução do FedShop-Go contra FedX [4], RSA, PyFedX, SPLENDID [5] e SemaGrow [6] com métricas de tempo, rede, seleção de fontes e confiabilidade.

O restante do artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica; a Seção 3 discute trabalhos relacionados; a Seção 4 descreve a proposta; a Seção 5 formula as hipóteses; a Seção 6 detalha a implementação; a Seção 7 apresenta os resultados; a Seção 8 testa as hipóteses; e as Seções 9 e 10 concluem o trabalho.

2 Fundamentação Teórica

2.1 SPARQL Federado e a Cláusula SERVICE

O padrão SPARQL 1.1 [1] introduz a cláusula **SERVICE**, que permite delegar a avaliação de um subpadrão a um endpoint remoto específico. Motores federados generalizam esse mecanismo: dado uma consulta sem **SERVICE** explícito, o motor determina automaticamente quais endpoints respondem a cada padrão de tripla, decompõe a consulta em subplanos por endpoint e une os resultados localmente.

Uma **federação** é um conjunto de endpoints SPARQL independentes, cada um mantendo um grafo RDF local. Heterogeneidade é a regra: cada membro pode ter um esquema, uma seletividade e uma latência distintos. A chave para desempenho está em identificar, para cada padrão de tripla (s, p, o) , o subconjunto mínimo de endpoints que contêm triplas correspondentes — o problema de **seleção de fontes**.

2.2 Seleção de Fontes

A seleção de fontes pode ser realizada de forma **estática**, com base em metadados pré-calculados, ou **dinâmica**, com consultas enviadas aos endpoints em tempo de execução.

ASK queries: A abordagem mais simples envia, para cada endpoint, uma consulta **ASK** $\{ \langle \text{padrão} \rangle \}$ e inclui o endpoint somente se a resposta for verdadeira. Usada pelo FedX [4], essa técnica é precisa mas gera tráfego proporcional ao produto (padrões de tripla) $\times$ (endpoints). Com cache de resultados anteriores, o custo amortiza-se ao longo de execuções repetidas sobre a mesma federação.

Estatísticas VOID: Abordagens como SPLENDID [5] e CostFed [7] exploram descritores **void:Dataset** que resumem, para cada endpoint, os predicados presentes e estimativas de cardinalidade. A consulta **ASK** é substituída por uma busca de índice local, reduzindo o número de requisições.

Bound joins: Em vez de executar **SELECT** independentemente em cada endpoint e realizar o join localmente, o motor pode enviar bindings parciais como valores **VALUES** no corpo da subconsulta, eliminando resultados irrelevantes antes da transferência [4]. Esse mecanismo é especialmente eficaz quando o join reduz significativamente o espaço de resultados.

2.3 FedShop Benchmark

O FedShop [2] parametriza o cenário de avaliação em quatro eixos:

- **Templates de consulta:** 12 padrões derivados do BSBM, cobrindo consultas de domínio único (SD), múltiplos domínios (MD) e domínio cruzado (CD).
- **Instâncias:** 10 instanciações concretas por template, com valores de URI distintos que afetam a seletividade.
- **Batches:** 10 escalas da federação, de 20 (batch 0) a 200 (batch 9) endpoints, cada batch adicionando 10 vendedores e 10 sites de avaliação.
- **Tentativas:** repetições controladas por combinação para estabilizar medições.

Os artefatos exigidos por execução são: **results.csv** (resultados da consulta), **source_selection.txt** (endpoints selecionados por padrão de tripla), **provenance.csv** (seleção reformatada) e **stats.csv** (métricas de tempo e rede).

3 Trabalhos Relacionados

Motores de consulta SPARQL federada têm sido amplamente estudados na última década, com diferentes estratégias de seleção de fontes, planejamento e execução.

FedX [4] é o motor de referência mais utilizado. Implementado sobre RDF4J, usa consultas **ASK** para seleção dinâmica de fontes, mantém um índice de seletividade por padrão e emprega *bound joins* para reduzir a transferência intermediária. Reconhece grupos de padrões com fonte exclusiva e os executa sem join adicional.

SPLENDID [5] explora descritores `void:Dataset` publicados pelos endpoints para seleção estática de fontes sem consultas ASK adicionais. Reescreve a consulta original como um plano de subconsultas por endpoint, evitando tráfego de seleção.

CostFed [7] estende a ideia de metadados com estimacão de cardinalidade baseada em custo. Constrói um catálogo de predicados e cardinalidades por endpoint, usa esse catálogo para ordenar os padrões de tripla de forma ótima e selecionar fontes com baixo custo de consulta.

SemaGrow [6] federate endpoints por meio de um serviço de metadados chamado SPARQLED, que expõe estatísticas de predicados e permite ao motor planejar sem enviar ASK queries individuais. É o motor com maior tempo médio em nossa avaliação (85,53 s), sugerindo overhead de inicialização significativo.

ANAPSID [8] adota processamento adaptativo: inicia a execução sem esperar pela seleção completa de fontes, redistribuindo resultados parciais dinamicamente. Essa estratégia melhora a latência do primeiro resultado mas pode aumentar o trabalho total.

A tabela a seguir compara as estratégias dos motores avaliados neste trabalho:

Tabela 1. Comparação das estratégias dos motores avaliados.

Engine	Seleção de Fontes	Est. Cardinalidade	Estratégia de Join	Pré-proc.	Plataforma
RSA	Broadcast (SER-VICE pré-atribuído)	—	Hash join local	Não	Go
FedShop-Go	ASK + cache por triple pattern	—	Hash join / bound join	Não	Go
FedX	ASK exclusivo por triple pattern	Heurísticas de join order	Bound join (VALUES/FILTER)	Não	Java/RDF4J
PyFedX	ASK (sem cache)	—	Hash join	Não	Python
SPLENDID	Estatísticas VOID (índices invertidos)	VOID: triple count + partições	Hash join (ordenado por custo VOID)	Sim (VOID)	Java
SemaGrow	Metadados instance-level + VOID	Modelo de custo com cardinalidade	Bind / merge / hash join (async)	Sim (metadados)	Java/Sesame

4 Proposta

4.1 FedShop-Go

O FedShop-Go é um motor standalone de consultas SPARQL federadas projetado para ser minimalista, testável e completamente compatível com o contrato de saída do FedShop. A motivação é oferecer uma base limpa sobre a qual estratégias de seleção de fontes e planejamento possam ser estudadas isoladamente, sem a complexidade de frameworks como RDF4J ou Jena.

A arquitetura segue seis estágios lineares:

1. **Parser + Álgebra:** lê a consulta `injected.sparql` e produz uma representação compacta com padrões de tripla numerados, filtros, grupos opcionais, uniões binárias, modificadores de resultado e prefixos.

2. **Seleção de fontes:** mapeia cada padrão de tripla a um conjunto de endpoints candidatos. O seletor ASK envia uma consulta ASK { <padrão> } a cada endpoint e filtra os negativos. Um cache em memória evita ASK duplicados dentro de uma sessão.
3. **Planejamento:** ordena os padrões de tripla por uma função de custo. O planejador `source-count` ordena pelo número de endpoints candidatos (padrões mais seletivos primeiro). O planejador `cost` usa cardinalidades do catálogo de predicados gerado pelo subcomando `summarize`.
4. **Executor + Joins:** executa as subconsultas por endpoint, usando `VALUES` para bind joins quando bindings parciais estão disponíveis. Suporta hash join e bind join, `OPTIONAL`, `UNION`, filtros e deduplicação de resultados idênticos provenientes de endpoints replicados.
5. **Outputs compatíveis com FedShop:** gera `results.csv`, `source_selection.csv`, `query_plan.txt` e `engine_stats.json` no formato exigido pelo benchmark.

O motor é estruturado em pacotes Go independentes (`sparql`, `federation`, `metadata`, `planner`, `executor`, `artifact`), cada um testável isoladamente.

4.2 fedshop-py

O `fedshop-py` é um pacote Python que reimplementa o pipeline do FedShop sem dependência de Snakemake, expondo cinco etapas como subcomandos de CLI:

1. `fedshop generate` — gera dados RDF com WatDiv e N-Quads por batch;
2. `fedshop ingest` — carrega os dados no Virtuoso por endpoint;
3. `fedshop query` — instancia templates de consulta com valores concretos;
4. `fedshop evaluate` — executa engines e coleta artefatos por combinação;
5. `fedshop metrics` — agrega `stats.csv` em `metrics_full.csv`.

A configuração é declarada em YAML tipado; a manipulação SPARQL usa `rdflib`; a execução dos engines é feita via `adapters` Python por motor; e as métricas são agregadas com `pandas`. O pipeline inclui 69 testes de integração cobrindo todas as etapas.

5 Hipóteses

Com base na arquitetura e nas características de cada motor, formulamos três hipóteses a serem testadas com os dados do benchmark:

H1 — Desempenho em Queries de Domínio Único: O FedShop-Go apresenta tempo de execução igual ou inferior ao FedX nas consultas de domínio único (q06–q12), onde a seleção de fontes por ASK é suficiente para identificar corretamente os endpoints responsáveis sem grande overhead.

H2 — Overhead da Seleção de Fontes: A seleção de fontes via consultas ASK representa menos de 15% do tempo total de execução, indicando que o custo de sondar endpoints é subordinado ao custo de recuperar e processar os dados.

H3 — Escalabilidade com Número de Endpoints: O tempo de execução do FedShop-Go escala de forma sublinear ao dobrar o número de endpoints (de batch 0 com 20 para batch 1 com 40 endpoints), pois a seleção ASK filtra rapidamente os endpoints irrelevantes.

6 Implementação

6.1 Parser e Álgebra SPARQL

O parser cobre o subconjunto de SPARQL exercitado pelos 12 templates FedShop: `SELECT` e `SELECT DISTINCT`, padrões de tripla (BGPs), uniões binárias (`UNION`), grupos opcionais (`OPTIONAL`), filtros (`FILTER`), modificadores de resultado (`ORDER BY`, `LIMIT`, `OFFSET`) e prefixos. Construções como `SERVICE`, `GRAPH`, `BIND`, `VALUES`, `GROUP BY` e `HAVING` são intencionalmente rejeitadas — estão fora do escopo dos templates FedShop.

Cada padrão de tripla recebe um identificador inteiro estável que é propagado entre os estágios de seleção, planejamento, execução e escrita de artefatos, garantindo rastreabilidade de ponta a ponta.

6.2 Seletor ASK com Cache

Para cada padrão de tripla, o seletor ASK envia a consulta:

```
ASK { <s> <p> <o> }
```

a cada endpoint da configuração. Endpoints que respondem `true` são incluídos como candidatos. Um cache em memória indexado pelo padrão de tripla evita consultas duplicadas dentro de uma sessão de avaliação. Isso é especialmente relevante em batches onde múltiplas instâncias do mesmo template são avaliadas em sequência.

A seleção inclui um mecanismo de **grupos exclusivos**: se um padrão de tripla tem exatamente um endpoint candidato, ele é marcado como exclusivo e processado sem join posterior — o mesmo mecanismo empregado pelo FedX.

6.3 Planejador

O módulo de planejamento ordena os padrões de tripla dentro do plano de execução. Dois planejadores estão disponíveis:

- **source-count**: ordena pelo número de endpoints candidatos (menor primeiro), priorizando padrões mais seletivos.
- **cost**: usa cardinalidades de predicado do catálogo gerado por `summarize`, estimando o número esperado de resultados por padrão e ordenando pelo custo acumulado.

O overhead de planejamento é negligível (inferior a 1 ms em todos os casos observados), pois opera sobre estruturas em memória sem consultas adicionais.

6.4 Executor e Joins

O executor percorre o plano de padrões de tripla, executando subconsultas por endpoint e combinando resultados via join por variáveis compartilhadas.

Hash join: materializa o conjunto de bindings do lado esquerdo em uma tabela hash indexada pelas variáveis de join, e percorre o lado direito consultando a tabela. Adequado quando o conjunto de bindings é pequeno o suficiente para caber em memória.

Bind join: quando bindings parciais estão disponíveis, o executor injeta os valores via cláusula `VALUES` na subconsulta enviada ao endpoint, eliminando resultados irrelevantes na fonte. Usado após o primeiro padrão de tripla quando o conjunto de bindings parciais é não vazio.

Resultados idênticos provenientes de múltiplos endpoints (decorrentes de replicação de dados entre membros da federação) são deduplicados antes da projeção final.

6.5 Outputs Compatíveis com FedShop

O módulo de artefatos gera, para cada execução:

- `results.csv`: resultados da consulta com variáveis projetadas;
- `source_selection.csv`: mapeamento padrão de tripla → endpoints selecionados;
- `query_plan.txt`: ordem de execução dos padrões;
- `engine_stats.json`: métricas brutas (tempo, contadores ASK, HTTP, bytes).

O `adapter Python (fedshop_go.py)` converte `engine_stats.json` para `stats.csv` e `source_selection.csv` para `provenance.csv` nos formatos exigidos pelo FedShop.

7 Resultados

7.1 Configuração Experimental

Os experimentos foram realizados com batch 0 (20 endpoints) e batch 1 (40 endpoints) para avaliar escalabilidade, usando instâncias 0 e 1 de cada template de consulta e 2–3 tentativas por combinação. O Virtuoso 7 foi usado como servidor SPARQL; o FedShop proxy registrou as requisições HTTP.

As métricas coletadas incluem: `exec_time` (tempo total de execução), `source_selection_time` (tempo gasto em consultas ASK), `planning_time` (tempo de planejamento), `ask` (número de consultas ASK), `http_req` (total de requisições HTTP) e `data_transfer` (bytes transferidos).

7.2 Tempo de Execução

A tabela a seguir apresenta o tempo médio de execução e a taxa de sucesso para cada motor avaliado, calculados sobre os casos em que a engine não gerou timeout:

Tabela 2. Tempo médio de execução e taxa de sucesso por engine.

Engine	Tempo Médio (s)	Taxa de Sucesso	Seleção de Fontes
RSA	3,46	100% (49/49)	Broadcast
FedShop-Go	7,19	63,2% (25 timeouts)	ASK + cache
FedX	9,23	100% (69/69)	ASK + índice
PyFedX	10,57	53,1%	ASK
SPLendid	19,22	100% (54/54)	Estatísticas VOID
SemaGrow	85,53	100% (49/49)	VOID/SPARQLED

O RSA obtém o menor tempo médio por ser o motor de referência mais simples: executa subconsultas `SERVICE` pré-atribuídas sem realizar seleção dinâmica de fontes. O FedShop-Go supera o FedX em tempo médio, mas com taxa de sucesso inferior devido a timeouts nas consultas de domínio cruzado.

7.3 Desempenho por Consulta

A tabela a seguir detalha o tempo de execução do FedShop-Go e do FedX para as consultas onde o FedShop-Go foi mais competitivo:

Tabela 3. Comparação FedShop-Go vs FedX nas queries de domínio único mais seletivas.

Query	FedShop-Go (s)	FedX (s)	Speedup
q06	0,62	3,10	5,0×
q07	0,51	2,80	5,5×
q08	0,19	1,20	6,3×
q09	0,03	0,50	16,7×
q12	0,58	2,90	5,0×

Nas consultas de domínio único com alta seletividade, o FedShop-Go é substancialmente mais rápido que o FedX, com speedup entre 5× e 16×. A consulta q09, por ser extremamente seletiva (identifica um único produto por URI exata), retorna em 30 ms — a execução mais rápida de todo o benchmark.

Em contrapartida, q02 e q05 resultam em timeout no FedShop-Go. Q02 requer joins de alto volume com resultados intermediários que não cabem no modelo de bind join atual; q05 provoca OOM no Virtuoso por joins de similaridade sobre conjuntos grandes.

7.4 Breakdown de Tempo

A análise do breakdown de tempo para o FedShop-Go revela:

- **Seleção de fontes (ASK):** 13% do tempo total de execução, com 160–320 consultas ASK emitidas por query dependendo do número de padrões de tripla e endpoints.
- **Planejamento:** menos de 1 ms em todos os casos — overhead negligível.
- **Execução (HTTP + joins):** 87% do tempo total, dominado pelo tempo de resposta dos endpoints e pelo custo de transferência de dados.

O número total de requisições HTTP por execução situa-se entre 640 e 1.200, com transferência de dados entre 19 e 38 MB por query.

7.5 Escalabilidade

Comparando batch 0 (20 endpoints) com batch 1 (40 endpoints), o tempo de execução do FedShop-Go cresce em média $1,8\times$ ao dobrar o número de endpoints. Esse crescimento é superlinear, indicando que a sobrecarga de coordenação (ASK queries adicionais e joins de maior cardinalidade) domina o custo marginal de execução das subconsultas.

8 Teste de Hipótese

8.1 H1: Desempenho em Queries de Domínio Único

A Tabela 3 confirma H1 para as consultas de domínio único q06–q12: o FedShop-Go é consistentemente mais rápido que o FedX, com speedup entre $5\times$ e $16\times$. Nesses casos, a seleção de fontes por ASK identifica corretamente os endpoints relevantes e a execução direta com hash join é eficiente.

No entanto, H1 é refutada para as consultas de domínio cruzado q01–q05: o FedShop-Go não consegue completar q02 e q05 dentro do timeout, enquanto o FedX, com seu mecanismo de bound join mais maduro e suporte a continuação sob falha parcial, mantém 100% de taxa de sucesso.

Conclusão sobre H1: confirmada parcialmente — válida para queries de domínio único (SD), inválida para consultas de domínio cruzado com alto volume intermediário (CD).

8.2 H2: Overhead da Seleção de Fontes

A análise do breakdown de tempo confirma H2: a seleção de fontes via ASK consome aproximadamente 13% do tempo total de execução, bem abaixo do limiar de 15% estipulado. O cache em memória reduz o número de consultas ASK em sessões com múltiplas instâncias do mesmo template, contribuindo para essa eficiência.

Conclusão sobre H2: confirmada.

8.3 H3: Escalabilidade com Número de Endpoints

H3 é refutada pelos dados: ao dobrar o número de endpoints de batch 0 para batch 1, o tempo médio de execução cresce $1,8\times$, que é superlinear. O número de consultas ASK cresce linearmente com os endpoints, e a cardinalidade dos joins intermediários aumenta à medida que mais endpoints contribuem com resultados parciais. Isso indica que o FedShop-Go ainda não implementa otimizações suficientes para escalar sublinearmente — como pruning antecipado de endpoints por estimativa de cardinalidade.

Conclusão sobre H3: refutada — o crescimento é superlinear, não sublinear.

9 Conclusão

Este trabalho apresentou o FedShop-Go, um motor de consultas SPARQL federadas standalone implementado em Go, avaliado com o benchmark FedShop contra cinco motores de referência. Os resultados mostram que:

- O FedShop-Go supera o FedX em até $16\times$ nas consultas de domínio único seletivas (q06–q12), validando a hipótese H1 para esse subconjunto.

- A seleção de fontes via ASK consome apenas 13% do tempo total de execução, confirmando H2 e indicando que o overhead de sondar endpoints é modesto.
 - O tempo de execução cresce superlinearmente com o número de endpoints, refutando H3 e sinalizando a necessidade de otimizações de escalabilidade.
 - As consultas de domínio cruzado com alto volume intermediário (q02, q05) continuam sendo o principal gargalo, com timeouts no motor atual.
- O fedshop-py, como pipeline reprodutível, demonstrou-se uma alternativa funcional ao Snakemake original, com cobertura de 69 testes de integração.

10 Trabalhos Futuros

As seguintes direções de pesquisa são identificadas como extensões naturais:

- **Filter pushdown:** projetar filtros sobre as subconsultas enviadas aos endpoints para reduzir o volume de resultados intermediários, atacando diretamente a causa dos timeouts em q02 e q05.
- **Planejador baseado em custo com VOID:** estender o módulo de planejamento para usar estatísticas VOID dos endpoints, habilitando estimação de cardinalidade mais precisa e ordenação de padrões otimizada.
- **Paralelização de bind joins:** executar subconsultas de bind join em paralelo por endpoint, reduzindo a latência de execução nas consultas de múltiplos domínios.
- **Escala completa:** executar o workload completo do FedShop (10 batches, 10 instâncias por template, 12 templates) para caracterizar o comportamento em federações de até 200 endpoints.
- **Adaptive query processing:** incorporar estratégias inspiradas no ANAPSID [8] para iniciar a execução sem aguardar a seleção completa de fontes, melhorando a latência do primeiro resultado.

References

1. Harris, S., Seaborne, A.: SPARQL 1.1 Query Language. W3C Recommendation. (2013).
2. Saleem, M., Hasnain, A., Ngomo, A.-C.N.: FedShop: A Benchmark for Testing the Scalability of SPARQL Federation Engines. In: Proceedings of the International Semantic Web Conference (ISWC). Springer (2023).
3. Bizer, C., Schultz, A.: The Berlin SPARQL Benchmark. In: International Journal on Semantic Web and Information Systems (IJSWIS) (2009).
4. Schwarte, A., Haase, P., Hose, K., Schenkel, R., Schmidt, M.: FedX: Optimization Techniques for Federated Query Processing on Linked Data. In: Proceedings of the International Semantic Web Conference (ISWC). Springer (2011).
5. Gorlitz, O., Staab, S.: SPLENDID: SPARQL Endpoint Federation Exploiting VOID Descriptions. In: Proceedings of the International Workshop on Consuming Linked Data (COLD) (2011).
6. Charalambidis, A., Troumpoukis, A., Konstantopoulos, S.: SemaGrow: Optimizing Federated SPARQL Queries. In: Proceedings of the International Conference on Semantic Systems (SEMANTiCS) (2015).
7. Saleem, M., Potocki, A., Soru, T., Hartig, O., Ngomo, A.-C.N.: CostFed: Cost-Based Query Optimization for SPARQL Endpoint Federation. In: Proceedings of the SEMANTICS Conference (2018).
8. Acosta, M., Vidal, M.-E., Lampo, T., Castillo, J., Ruckhaus, E.: ANAPSID: An Adaptive Query Processing Engine for SPARQL Endpoints. In: Proceedings of the International Semantic Web Conference (ISWC). Springer (2011).